

E题题解

可以从 Hint 里得知，这道题的分治算法可见《算法导论》第三版39-41页，在此不赘述。

这里说明一种复杂度为 $O(n)$ 的贪心算法。

首先，定义两个变量，一个存最大值，一个存临时用来运算的值。

```
long long int sum=-2000000000;  
long long int temp;
```

大致思路是，**无论加上当前数是否让 temp 的值变小，都接受**，然后与之前的最大值 sum 比较。如果大于最大值，则**更新最大值**。

那么什么时候截断 temp 的运算？**遇到负数就直接截断是不行的**，也许之前的加和可以中和掉负数带来的负面影响，并给之后的数以助力，例如：

```
{2,7,-5,6}
```

最大子数组不应当是 $2+7=9$ ，而应当是 $2+7+-5+6=10$ 。所以，就形成了下述的思路：

此题需要寻找最大子数组，那么每遍历一个数，都会希望**之前的运算结果，对于即将遍历的这个数，起到正面作用**。否则，**不如舍弃之前的部分**。

那么，**每当 temp 的值小于0时，就把它更新为0**，因为一个负数加之后的数，一定小于直接将之后的数存入 temp，从而达到寻找最大子数组的目的。

关键代码如下：

```
for(i=0;i<n;i++)  
{  
    temp+=cal[i];  
    if(sum<temp)  
        sum=temp;  
    if(temp<0)  
        temp=0;  
}
```

完整代码如下：

```

#include<stdio.h>

long long int sum=-2000000000;
long long int temp;
int cal[2000000];
int main()
{
    int n;
    scanf("%d",&n);
    int i;
    for(i=0;i<n;i++)
        scanf("%d",&cal[i]);

    for(i=0;i<n;i++)
    {
        temp+=cal[i];
        if(sum<temp)
            sum=temp;
        if(temp<0)
            temp=0;
    }

    printf("%lld\n",sum);
    return 0;
}

```

THE END